

1. Motivation & Problem

A mobile robot follows a global route with a **local controller** that decides its behaviour cycle-by-cycle.

- **Classical samplers** (DWB, MPPI) score trajectories with a *weighted sum of interpretable costs* — transparent and safe, but the weights are *fixed* at tuning time and one weighting cannot suit every context.
- **End-to-end learned** controllers adapt from data but are *opaque*; safety is only a learned regularity, not a guarantee.

Gap: a controller that adapts to context every cycle while keeping the hard safety of the classical design.

Objective. Separate *what is learned* from *what is controlled* — a context-conditioned network re-weights a classical controller's interpretable costs each cycle, while the planner retains trajectory feasibility and obstacle evaluation, making safety *structural* rather than learned.

Research question. Does learning only the per-cycle cost weighting outperform both fixed-weight classical planners and an end-to-end learned baseline on a multi-criteria aggregate, without sacrificing hard safety?

2. Approach — CA-MCW & Safety by Decomposition

Context-Adaptive Multi-Critic Weighting. Keep a classical **MPPI** planner and its bank of interpretable critics intact; *learn only their relative importance*. Each cycle a compact perception-conditioned network outputs non-negative multipliers $\mathbf{m}_t$ that rescale the configured critic weights:

$$w_{c,t}^{\text{eff}} = m_{c,t} \cdot w_c^o, \quad m_{c,t} \geq 0.05.$$

Trajectory generation, feasibility and obstacle evaluation *stay inside the planner*, so even a pathological network yields at worst a *sub-optimal but still feasible* trajectory.

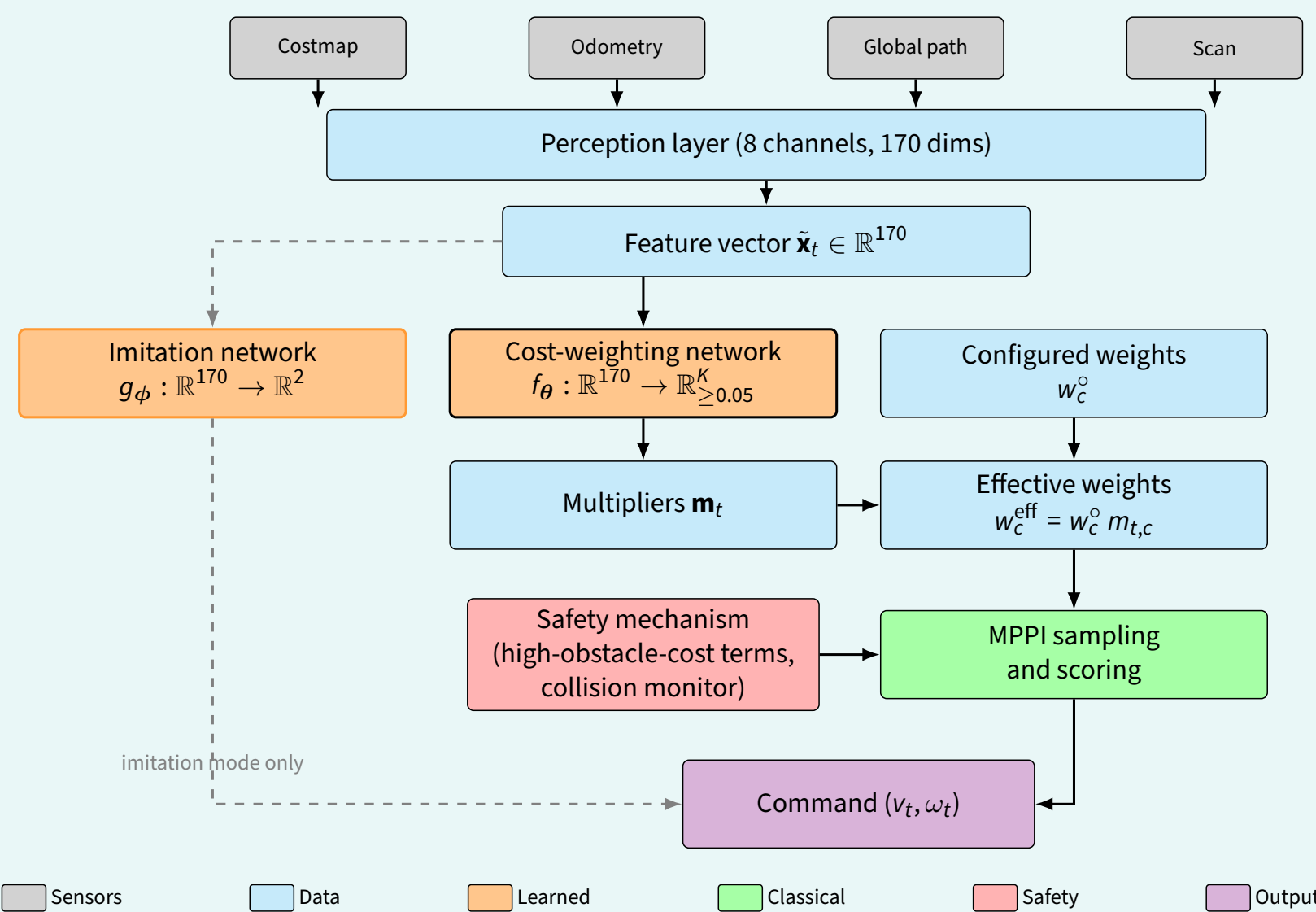


Fig. 1 High-level pipeline; only the orange block is learned.

3. Perception Input — 170-D Feature Vector

A single 170-dimensional vector, partitioned into eight named channels (three provenance families), conditions the network each cycle.

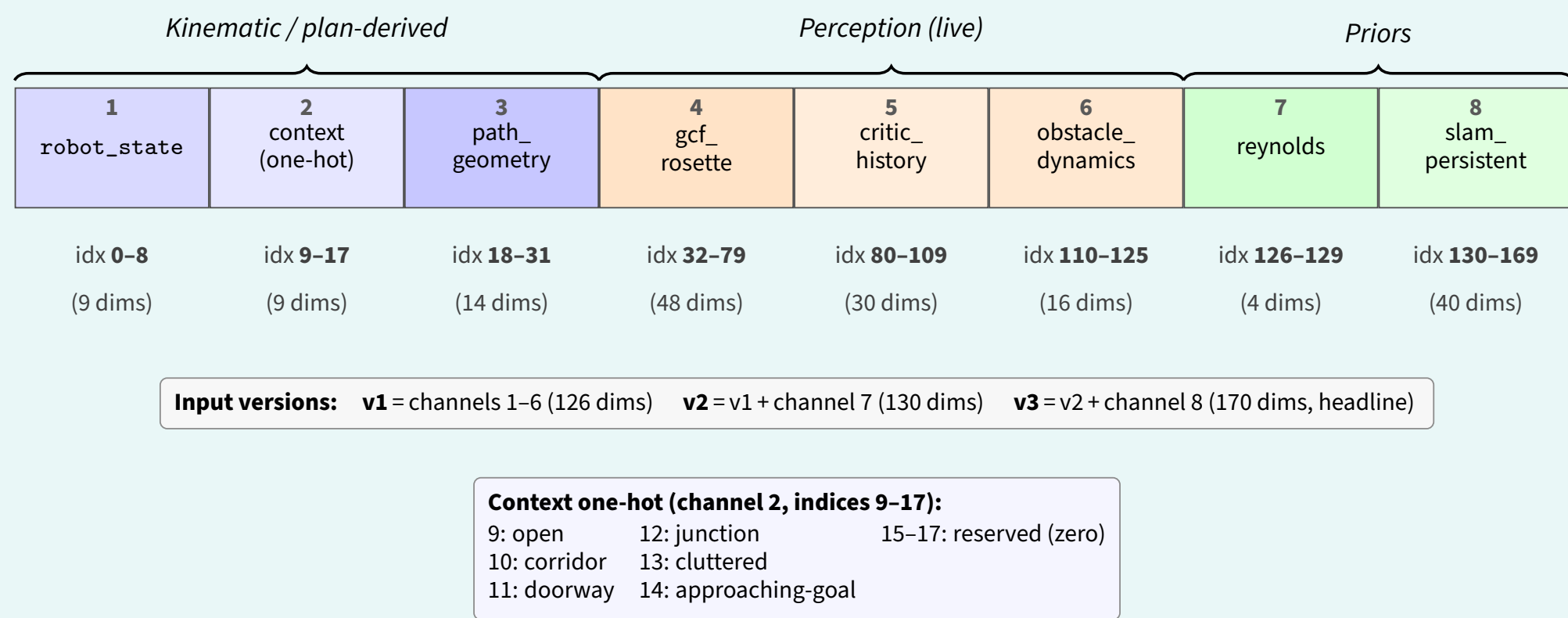


Fig. 2 Index-range layout of the v3 perception vector $\mathbf{x}_t$.

4. Cost-Weighting Network

A channel-wise frontend feeds a (256, 128, 64) fusion stack and a paradigm-specific head: 64 → 11 + softplus for the multiplier paradigms, 64 → 2 for the imitation baseline.

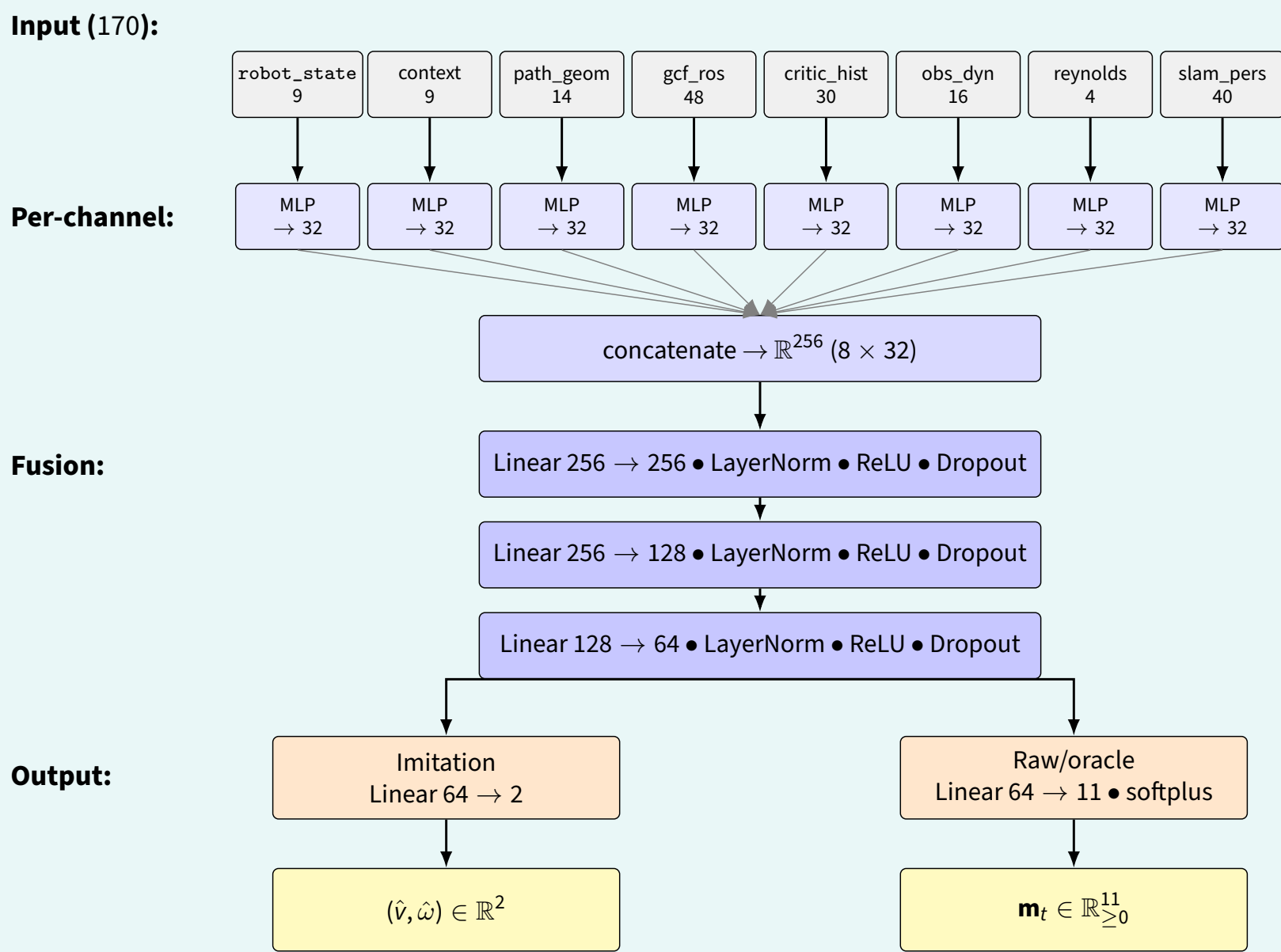


Fig. 3 End-to-end structure of the learned controllers.

5. Algorithm & Critic Taxonomy

Algorithm 1 — CA-MCW control cycle

```
1:  $\mathbf{x}_t \leftarrow \text{assembleFeatures}(\text{costmap}, \text{odom}, \text{path}, \text{scan})$ 
2:  $\mathbf{m}_t \leftarrow \text{weightProvider}(\mathbf{x}_t)$ 
3: for cost term  $c \in \{1, \dots, K\}$  do
4:    $w_{c,t}^{\text{eff}} \leftarrow w_c^o \cdot m_{t,c}$ 
5: end for
6:  $\tau^* \leftarrow \text{MPPI}(\{w_{c,t}^{\text{eff}}\}_{c=1}^K)$ 
7:  $(\mathbf{v}_t, \omega_t) \leftarrow \tau^*[0]$ 
8: return  $(\mathbf{v}_t, \omega_t)$ 
```

The multiplier enters *only* the cost-weight product; feasibility and command

Reynolds-grouped critics

The ten cost terms are reorganised so each maps to a named, auditable behaviour from Reynolds' flocking primitives, adapted to single-robot navigation:

Primitive	Role
Separation	clear of obstacles / walls
Alignment	match heading to plan
Cohesion	stay on the global path
Goal-seeking	progress to the goal

Gives the learned weight vector a physically meaningful axis

6. Controlled Experimental Design

Feature representation, dataset and training procedure are held constant across **CA-MCW** (recorded per-cycle costs, *deployed*), an **oracle-weight** variant (convex cost-optimal weights) and an end-to-end **IL-V** imitation baseline. **23 controllers** drive the **same 8-goal tour** in an **unseen test environment**, ranked by **entropy-weighted TOPSIS**.

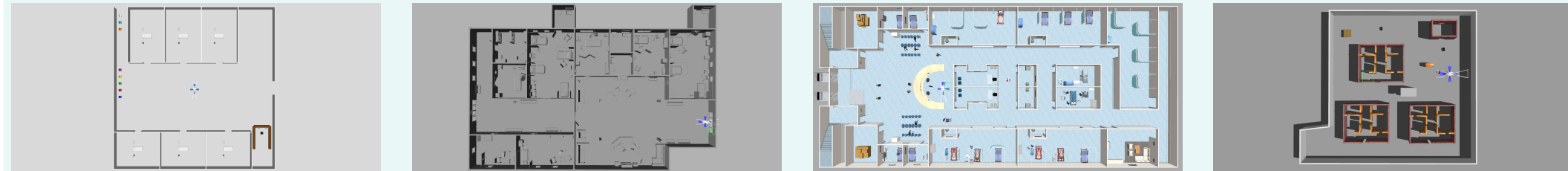


Fig. 4 Gazebo worlds: three training environments (left three) + the unseen test environment (right).

7. Training Convergence

One representative model per paradigm; curves drop monotonically and validation tracks training without divergence — early stopping fires before overfitting.

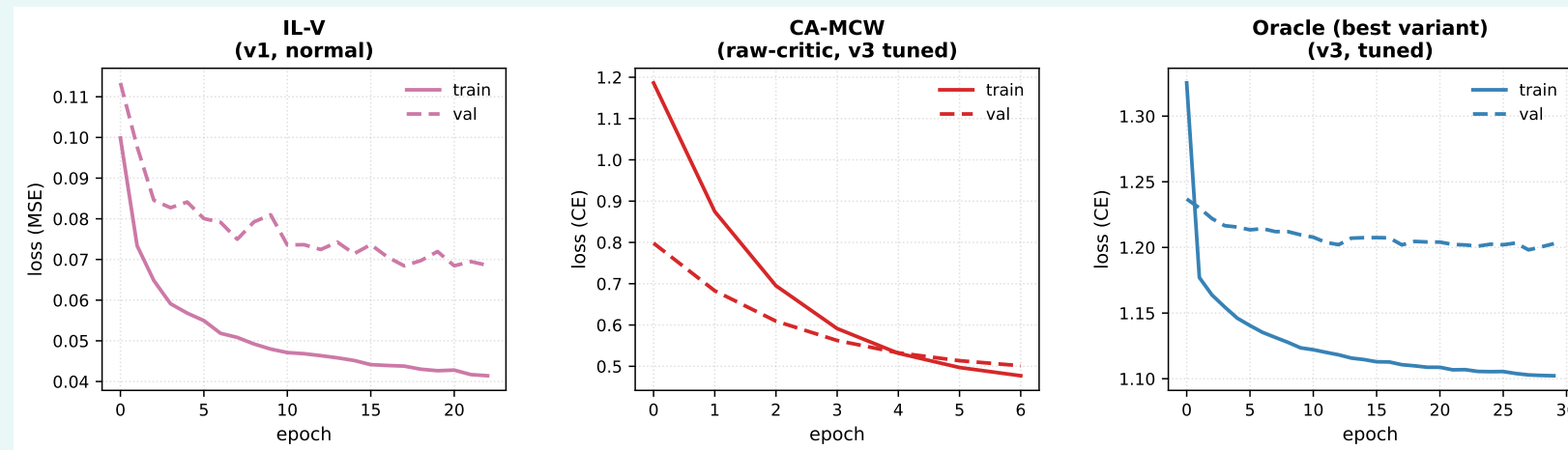


Fig. 5 Training / validation curves per paradigm.

8. Results — CA-MCW Ranks First

Evaluation tour — every goal, the fully-successful controllers (per-leg re-anchored to nominal start)

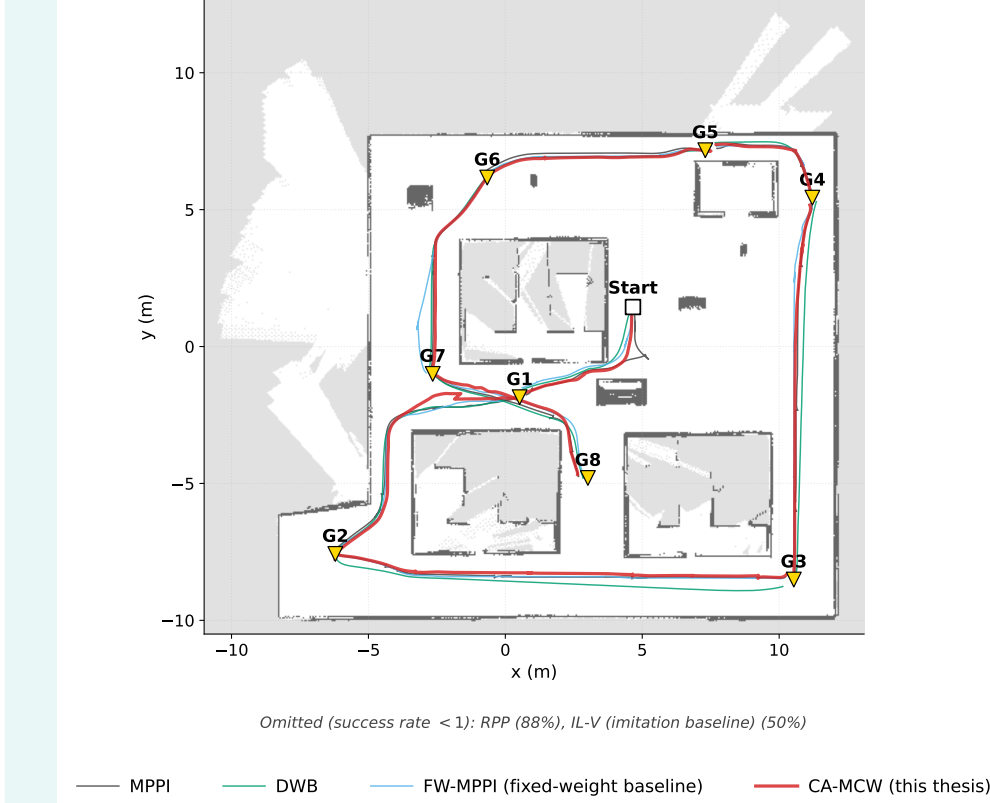


Fig. 6 Eight-goal evaluation tour (fully-successful controllers).

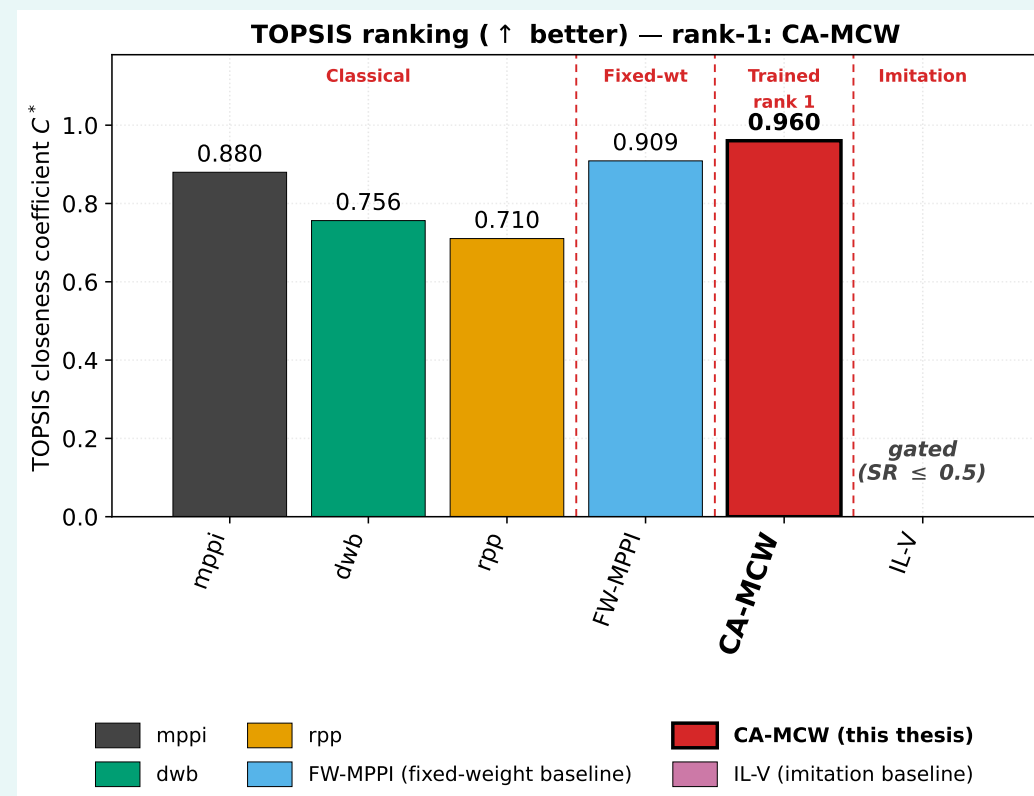


Fig. 7 TOPSIS closeness C^* (higher better).

Controller	C^*
CA-MCW	0.96
FW-MPPI	0.91
mppi	0.88
dwb	0.76
rpp	0.71
IL-V	gated

- **Zero collisions; 100% task completion** over the tour.
- Best worst-case clearance, smoothness, goal accuracy; concedes only raw speed.
- Outranks every baseline, FW-MPPI, *and* every oracle variant.
- **IL-V: only 4/8 goals** and the *only* collider.

9. Path Adherence

Cross-track error against each controller's own active plan: the five completing controllers track tightly (≈ 0.02 m); IL-V is the outlier.

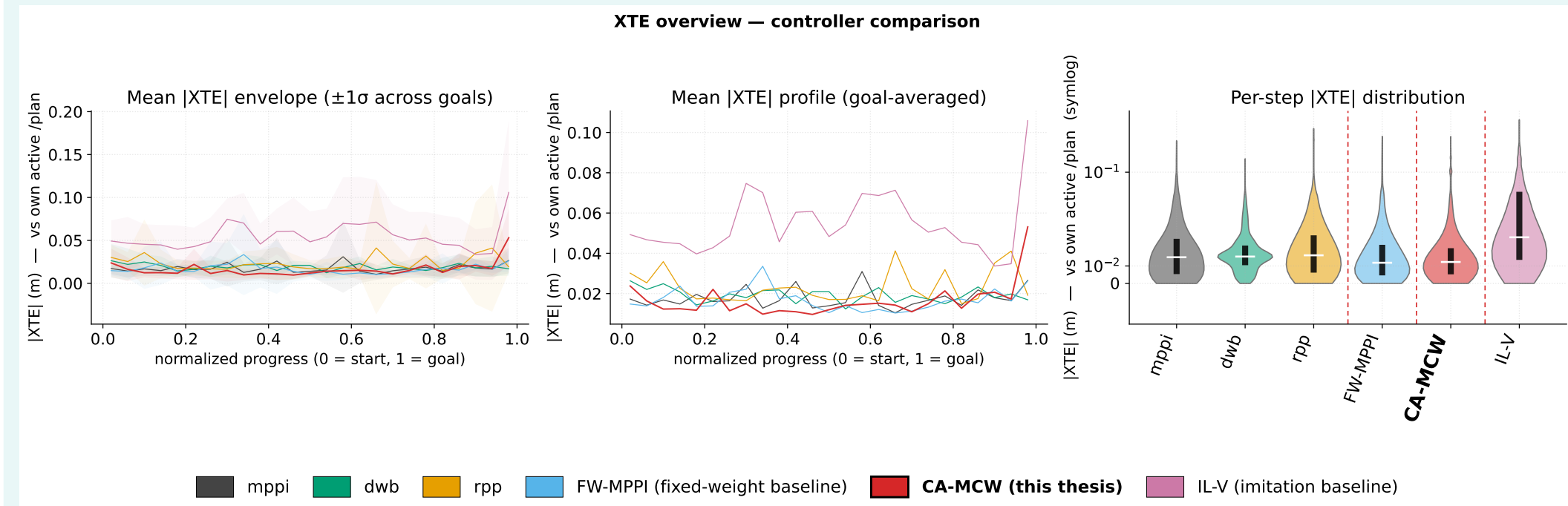


Fig. 8 Cross-track error vs. own active plan: $\pm 1\sigma$ envelope, goal-averaged profile, and per-step distribution.

10. Learned Adaptation Signature

Per-critic weight share varies *systematically* with obstacle proximity — a **context-conditioned schedule**, not a constant vector. It does *not* reproduce the sparse oracle, yet outranks the model trained to match it.

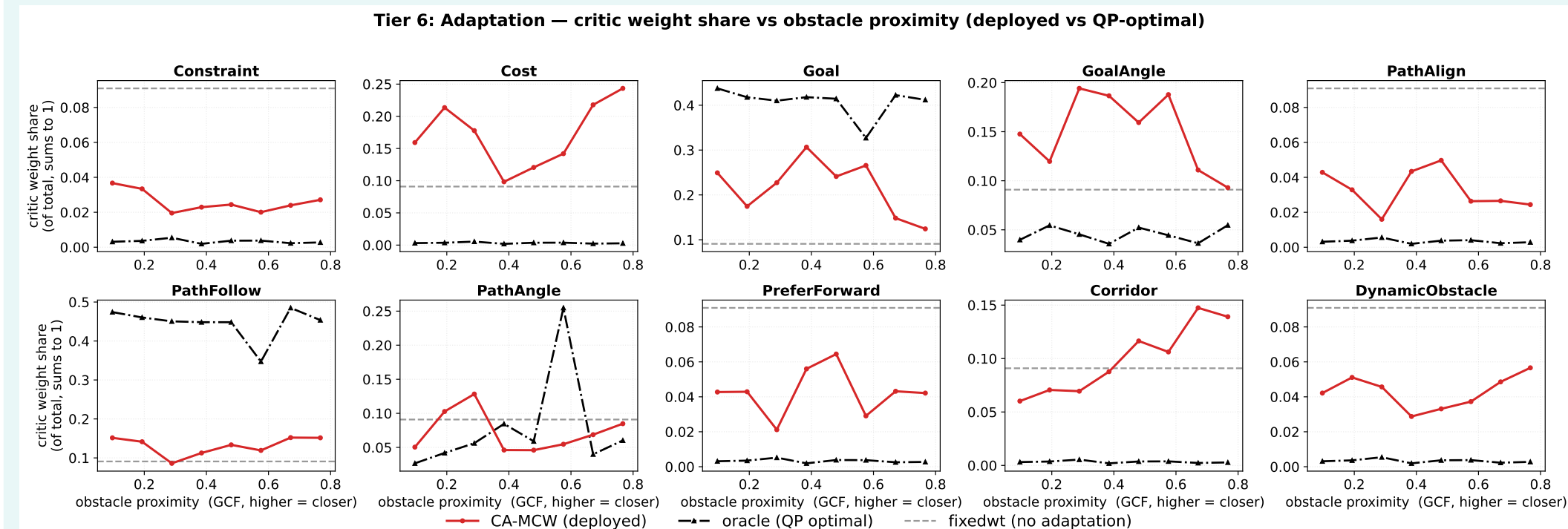


Fig. 9 Per-critic weight share vs. obstacle proximity (deployed vs. QP-optimal oracle vs. no-adaptation floor).

Adaptation signature: the deployed weights move *with context* — share migrates between critic groups as obstacle proximity changes, tracking the oracle's qualitative trend (not its sparse magnitudes); a single fixed weighting cannot express this.

11. Conclusion & Future Work

Conclusion. By decomposing the controller into a *learned* per-cycle weighting and a *classical* safety-bearing MPPI planner, CA-MCW gains context-adaptive behaviour without surrendering the hard guarantees of the sampling-based design — and tops every baseline, FW-MPPI and every oracle variant.

- **Sim-to-real:** deploy on the physical robot and quantify the reality gap.
- **Dynamic scenes:** moving obstacles and crowds beyond the static tour.
- **Joint learning:** train base weights w_c^o and multipliers $\mathbf{m}_t$ end-to-end rather than rescaling a fixed configuration.
- **Scaling:** larger critic banks and richer perception channels.

Separating *what is learned* (weighting) from *what stays classical* (safety) yields a controller that is both context-adaptive *and* safe by construction — the best-balanced of all 23 evaluated.